Python Basics

String Manipulation in Python

A **string** is text in Python. It can use single quotes, double quotes, or triple quotes. Strings help you store and work with words and sentences.

String Slicing Syntax

Slicing means taking a part of a string. Use `text[start:stop:step]` to choose which letters you want.

```
text = 'Python Programming'

text[0:6]   # Python
text[7:]    # Programming
text[0:12]  # Python Progr
```

1

start

Where to begin

Example: `text[0:6]` starts at 0.

2

stop

Where to end

Example: `text[0:6]` stops before 6.

3

step

How many letters to skip

Example: `text[::2]` takes every other letter.

Made with GAMMA

DYNAMIC OUTPUT

String Formatting

String formatting inserts variables into strings for readable, dynamic output. Python offers three main approaches — f-Strings are the recommended modern method.

f-Strings

Recommended

```
name = "Bob"
f"Hi {name}"
```

format()

Flexible

```
name = "Bob"
"Hi {}".format(name)
```

% formatting

Legacy

```
age = 25
"Age: %d" % age
```

Code Examples

```
name = "Bob"

f"Hi {name}"

"Hi {}".format(name)

age = 25
"Age: %d" % age
```

Made with GAMMA

TEXT PROCESSING

Common String Methods

Built-in methods help you work with text. Since strings are immutable, these methods return new strings instead of changing the original.

upper() & lower()

Change text to uppercase or lowercase.

```
text = "Hello"
print(text.upper())
# HELLO
```

replace() & split()

Swap words in text, or break text into parts.

```
text = "apple pie"
print(text.replace("pie", "juice"))
# apple juice
```

join() & count()

Put words together, or count how many times something appears.

```
words = ["red", "blue"]
print("-".join(words))
# red-blue
```

find() & strip()

Look for text, or remove extra spaces.

```
text = "  hello  "
print(text.strip())
# hello
```

These methods make it easier to clean, change, and check text.

Made with GAMMA